

분야	과정	과목명
컴퓨터공학	전공심화(3과정)	소프트웨어공학

대영역	중영역	소영역
1. 소프트웨어 공학 소개	가. 소프트웨어 공학이란	1. 소프트웨어 공학의 필요성 2. 소프트웨어 공학의 정의
	나. 소프트웨어의 특성	1. 소프트웨어의 특성 2. 소프트웨어의 분류 3. 좋은 소프트웨어의 기준
	다. 소프트웨어 위기 현상	
	라. 소프트웨어 개발 단계	
2. 소프트웨어 개발 프로세스	가. 프로세스 모델 개요	
	나. 선형순차(폭포수) 모델	
	다. 점증적 모델	
	라. 진화형 모델	
	마. 나선형 모델	
	바. V 모델	
	사. 통합 프로세스(UP) 모델	
	아. 애자일 방법	1. 애자일 원리와 지침 2. XP 3. 스크럼
3. 소프트웨어 프로젝트 관리	가. 프로젝트 관리의 중요성	
	나. 프로젝트 계획 수립	1. 소프트웨어 프로젝트 계획서 작성 2. 프로젝트 관리 도구
	다. 소프트웨어 프로젝트 비용 추정	1. 비용과 노력추정 모델 (경험적 추정, 델파이, 알고리즘 기법) 2. 규모 기반 모델(COCOMO) 3. 기능 점수(FP)
	라. 소프트웨어 프로젝트 일정 계획	1. 일정 계획의 이해 2. 업무 정의 및 분해(WBS) 3. 일정계획 방법(간트 차트, 네트워크 차트)
	마. 조직 계획	
	바. 위험 관리	1. 위험 분석의 이해

대 영역	중 영역	소 영역
4. 소프트웨어 품질		2. 위험의 종류
		3. 위험 관리 절차
	가. 소프트웨어 품질의 이해	1. 소프트웨어와 품질
		2. 품질 보증과 품질 비용
		3. 품질 관점과 품질 목표
	나. 제품의 품질 특성과 평가 모델	1. 맥콜(McCall)의 품질 요소
		2. ISO/IEC 9126 모델
		3. ISO/IEC 25000 모델
	다. 프로세스 품질 모델과 개선 모델	1. ISO 12207 모델과 프로세스 표준
		2. ISO 9000과 품질 경영 시스템
		3. SPICE(ISO 15504) 모델
		4. CMMI 모델
	라. 품질 보증 활동	1. 품질 보증 조직
		2. 품질 제어
		3. 확인과 검증
		4. 정적 검토(동료 검토, 검토 회의, 인스펙션)
	마. 소프트웨어 신뢰도	1. 소프트웨어 결함
		2. 가용성과 신뢰도 척도
5. 요구사항 분석	가. 요구사항의 이해	1. 요구 분석의 정의와 목적
		2. 요구사항의 중요성
		3. 요구사항의 추출
	나. 요구사항의 종류	1. 기능적 요구사항과 비기능적 요구사항
		2. 사용자 요구사항과 시스템 요구사항
	다. 요구 분석 절차	1. 요구 공학
		2. 도메인 분석
		3. 요구사항 검토와 관리
	라. 요구사항의 표현	1. 모델의 정의와 필요성
		2. 모델링 언어
		3. 요구 분석 명세서
	마. 구조적 분석	1. 구조적 방법
		2. 데이터 모델링과 개체-관계 다이어그램
		3. 기능 모델링과 데이터 흐름도

대영역	중영역	소영역
		4. 행위 모델링과 상태전이도
		5. 프로세스(소단위) 명세
		6. 데이터 사전
	바. 객체지향 분석	1. 요구사항 추출과 분석
		2. 유스케이스와 사용자 스토리
		2. 분석 객체 모델
		3. 동적 모델
6. 소프트웨어 설계	가. 소프트웨어 설계의 중요성	
	나. 설계 프로세스	1. 데이터 설계
		2. 아키텍처 설계
		3. 인터페이스 설계
		4. 프로시저 설계
	다. 설계 원리	1. 추상화
		2. 단계적 정제
		3. 모듈화
		4. 캡슐화와 정보은닉
		5. 분할과 정복
		6. 구조적 분할
		7. 상향식과 하향식 설계
	라. 구조적 설계	1. 구조도
		2. 변환 분석
		3. 트랜잭션 분석
	마. 아키텍처 설계	1. 아키텍처의 중요성
		2. 아키텍처와 품질 속성
		3. 아키텍처 설계 원리와 설계 목표
		4. 아키텍처 스타일
	바. 설계 평가	
	사. 설계 문서화	
7. 상세 설계	가. 모듈화	1. 기능적 독립성
		2. 응집도
		3. 결합도
	나. 클래스 설계	1. 클래스 인터페이스의 정의

대영역	중영역	소영역
		2. 클래스 설계 원칙
		3. 클래스 재사용
	다. 사용자 인터페이스 설계	1. 사용자 인터페이스 유형
		2. 사용자 인터페이스 설계 원리
		3. 화면 설계와 출력물 설계
	라. 디자인 패턴	
8. 구현	가. 코딩	
	나. 구조적 프로그래밍	
	다. 코딩 스타일 표준	
	라. 코드 문서화	
	마. 코드 스멜과 리팩토링	1. 코드 스멜
		2. 리팩토링
9. 소프트웨어 테스트	가. 소프트웨어 테스트의 이해	1. 테스트 원리
		2. 테스트케이스 설계
		3. 테스트 프로세스
	나. 단계별 테스트와 통합	1. 단위 테스트
		2. 통합 테스트(상향식, 하향식, 연쇄식, 빅뱅)
		3. 시스템 테스트
		4. 인수 테스트(알파, 베타)
		5. 회귀 테스트
	다. 화이트박스 테스트	1. 테스트 커버리지
		2. 논리 흐름 그래프
		3. 기본 경로 테스트
		4. 사이클로매틱 복잡도
	라. 블랙박스 테스트	1. 동치 분할
		2. 경계값 분석
		3. 원인결과 그래프
	마. 시스템 테스트	1. 회복 테스트
		2. 보안 테스트
		3. 강도 테스트
		4. 성능 테스트
	바. 테스트 자동화	1. 테스트 도구
		2. 테스트와 디버깅
10. 소프트웨어 유지보	가. 소프트웨어 유지보수 개요	1. 레거시 시스템

대영역	중영역	소영역
수		2. 유지보수 정의
		3. 유지보수 전략
	나. 유지보수 프로세스	1. 유지보수 유형과 비용
		2. 시스템의 진화 과정
		3. 유지보수 프로세스 모델
	다. 재공학	1. 순공학과 역공학
		2. 재공학프로세스
		3. 재구조화, 모듈화, 데이터 재공학
	라. 역공학	1. 역공학 프로세스
		2. 설계 복구와 재문서화
		3. 유지보수 부작용(코드, 데이터, 문서)
	마. 형상 관리	1. 형상 관리 필요성
		2. 형상 항목과 베이스라인
		3. 형상 관리 활동
	바. 유지보수성	1. 유지보수성의 제어요인
		2. 유지보수성의 정량적 척도
		3. 유지보수 도구
11. 객체지향 개발과 UML	가. 객체지향 개발 생명주기	1. 객체지향 원리
		2. 소프트웨어 개발과 모델링
		3. 객체지향 개발 프로세스
	나. UML의 개념	
	다. 유스케이스 모델링	1. 유스케이스 분석
		2. 액터와 유스케이스
		3. 유스케이스 관계
		4. 유스케이스 명세
	라. 클래스 모델링	1. 객체와 클래스
		2. 클래스의 표현
		3. 연관(association) 관계
		4. 추상 클래스와 인터페이스
		5. 상속
		6. 클래스 다이어그램의 코딩
	마. 동적 모델링	1. 시퀀스 다이어그램

대 영역	중 영역	소 영역
		2. 상태 다이어그램
		3. 통신 다이어그램
		4. 액티비티 다이어그램
	바. 구조 모델링	1. 컴포넌트 다이어그램
		2. 배포 다이어그램
		3. 패키지 다이어그램
	사. 디자인 패턴	
12. 소프트웨어 공학의 최근동향	가. CASE	1. CASE 도구의 분류
		2. CASE 저장소
		3. 도구와 통합
		4. 모델링 도구
		5. 프로젝트 관리 도구
		6. 버전 관리 도구
	나. 소프트웨어 재사용	1. 재사용의 장점
		2. 재사용 도입의 문제점
		3. 재사용 프로세스
		4. 도메인 공학
		5. 재사용 가능 컴포넌트 구축
	다. 웹서비스와 SOA	1. SOA의 개요
		2. 웹서비스와 SOA의 구현
		3. SOA의 구조

시험 문제 예시

1. 폭포수 모델(waterfall model)에 대한 설명으로 옳은 것은?

- ① 점진적인 개발 모형이다.
- ② 각 단계의 구별이 명확하지 않다.
- ③ 요구사항이 명확하지 않을 경우에 사용한다.
- ④ 선형 순차 모델(linear sequential model)이라고도 부른다.

[정답] ④

2. 소프트웨어 설계 지침으로 옳지 않은 것은?

- ① 이식성을 고려한다.
- ② 결합도를 줄이고 응집도를 높인다.

- [정답] ③

- ① 모듈화를 촉진시키고, 일관성이 보장된다.
- ② 내부 데이터 변화에 대해 외부 객체에 영향이 없다.
- ③ 이식성과 확장성이 매우 높아진다.
- ④ 캡슐화된 내부 정보도 외부에서 직접적 접근이 가능하다.

[정답] ④

- ① 집약적이고 재사용성이 높다.
- ② 시스템 수정에 융통성이 높다.
- ③ 공동 작업하는 데는 다소 불편하다.
- ④ 실세계 묘사로 설계 내용을 이해하기 쉽다.

[정답] ③

① 추상화 ② 결합도
③ 응집도 ④ 기능적 독립성

[정답] ②

(㉠) 테스트는 프로그램이 수행하는 기능에 초점을 가지고 테스트하는 방법으로 프로그램의 내부 구조는 다루지 않는다. (㉡) 테스트는 모듈 안의 작동을 자세히 관찰하는 방법으로 모듈의 논리적인 구조를 체계적으로 점검한다.

[정답] ㉠ 블랙박스, ㉡ 화이트박스

[정답] 1. 프로그래밍 언어와는 독립적이어서 일반언어나 비절차적 언어를 이용한 응용

들에 대해 이상적이다.

2. 프로젝트 개발 초기에 쉽게 알려질 수 있는 데이터를 기초로 한다.